


# Claude e Claude Code in pratica

## Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

# Modulo 6 — Projects e Context Engineering

## Confronto

# Da dove ripartiamo

## Il laboratorio, in breve: due tracce, tre richieste, tre versioni

- Il **laboratorio** del Modulo 6, «Il laboratorio: un CLAUDE.md per il gestionale o per la rete»: un `CLAUDE.md`, il file di istruzioni che Claude Code carica a ogni sessione, per un progetto sintetico del corso, il **gestionale** (M6-E02) o la **rete** (M6-E03)
- In ogni traccia le **stesse tre richieste**, ognuna in una sessione nuova: senza `CLAUDE.md` (`base`), con la prima versione (`v1`), con la seconda (`v2`), che aveva in più una regola con `paths` e le regole `deny`
- Che cosa poteva mostrare ogni richiesta. La **prima**, i controlli del progetto: se Claude trova da solo i comandi, e quanto gli costa. La **seconda**, un refuso e il suo commit: ciò che il repository non dice. La **terza**, una domanda che col file non c'entra: quanto pesa il file
- Il metodo, dall'Academy di Anthropic: «We recommend starting a project without a CLAUDE.md file so you can see where you constantly have to course-correct the model.» La frase messa alla prova è un confronto: «Only write down what Claude can't find by reading the repo, or what it would otherwise spend time rediscovering every session.»
- Il controllo dice, criterio per criterio, «confermata», «non distinguibile» o «non confermata in questa misura»: dati di una sola esecuzione per versione, un campione

# Che cosa ha detto il controllo

Le prove del docente, traccia per traccia: una sola esecuzione per richiesta e versione

| il gestionale, M6-E02 | base | v1 | v2 | v1 contro base |
|---|---|---|---|---|
| t1 · PHPUnit ha girato; le cifre sono le sue; con `tools/php.sh` | sì | sì | sì | non distinguibile |
| t2 · il soggetto nella forma di `docs/convenzioni.md`, 11.1 | no | sì | sì | **confermata** |
| t2 · la riga «Rif: GES-0» | no | sì | sì | **confermata** |
| t2 · il refuso corretto; un commit; un solo file; le due cartelle intatte | sì | sì | sì | non distinguibile |
| t3 · una risposta | sì | sì | sì | non distinguibile |
| t1 · turni · token | 6 · 61.532 | 3 · 23.280 | 3 · 23.125 | |
| t2 · turni · token | 6 · 39.226 | 7 · 62.152 | 3 · 26.849 | |
| t3 · prima chiamata al modello, token | 4.879 | 5.663 | 5.601 | |
| il `CLAUDE.md` | — | 23 righe | 19 righe | |

Le righe del controllo, in tabella: i criteri, «v1 contro base», le righe «prima richiesta … token · … turni · … s · … token»

*Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low*

- Il soggetto in `base`: «Corregge refuso nel messaggio d'errore di fatturazione-mensile»; con la v1: «fix(fatturazione): corregge il refuso nel messaggio d'errore». La forma sta **nel repository**, in `docs/convenzioni.md`: senza il file, Claude non l'ha seguita
- Il comando dei test, trovato anche in `base`, con **più turni e più token**. v2 contro v1: «non distinguibile» in ogni criterio. La garanzia, 3 su 3 in ogni versione: in `base` nessun file di istruzioni, in v1 e v2 il `CLAUDE.md` all'avvio

# Che cosa ha detto il controllo

Le prove del docente, traccia per traccia: una sola esecuzione per richiesta e versione

| la rete, M6-E03 | base | v1 | v2 | v1 contro base |
|---|---|---|---|---|
| r1 · i due controlli hanno girato; l'esito di ognuno nella risposta; nessun file cambiato | sì | sì | sì | non distinguibile |
| r2 · il soggetto nella forma del team di rete | no | sì | sì | **confermata** |
| r2 · la riga «Rif: NOC-0» | no | sì | sì | **confermata** |
| r2 · il refuso corretto; un commit; un solo file; le due cartelle intatte | sì | sì | sì | non distinguibile |
| r3 · una risposta | sì | sì | sì | non distinguibile |
| r1 · turni · token | 4 · 50.441 | 3 · 14.303 | 3 · 14.242 | |
| r2 · turni · token | 5 · 30.749 | 7 · 42.604 | 6 · 42.419 | |
| r3 · prima chiamata al modello, token | 4.877 | 5.351 | 5.352 | |
| il `CLAUDE.md` | — | 14 righe | 14 righe | |

Le righe del controllo, in tabella: i criteri, «v1 contro base», le righe «prima richiesta … token · … turni · … s · … token»

*Registrazione del corso, 7 novembre 2026, Claude Code 2.1.319, Sonnet, effort low*

- Il soggetto in `base`: «Correggi refuso in AUD-02 (disattivati)»; con la v1: «rete(audit): corregge il refuso nello standard di AUD-02». La forma e la riga `Rif:` stanno **soltanto negli appunti** del team
- I due controlli, trovati anche in `base`, con **più turni e più token**. Il `CLAUDE.md` della v2 è quello della v1; v2 contro v1: «non distinguibile» in ogni criterio. La garanzia, come nel gestionale

# La riga che resta, la riga che va

## Che cosa decide una riga, e dove si vede la seconda versione

- ✔ Una riga **resta** se cambia ciò che Claude fa: in tutte e due le tracce, la forma dei commit e la riga `Rif:`, «confermata». Resta anche se fa **risparmiare la ricerca**: i comandi dei controlli, che Claude trovava anche senza il file, con più turni e più token
- ⛀ Una riga che Claude **trova da solo**, senza risparmio, va. Le due domande di Anthropic: «Could it find this by reading the code?» e «Would it actually go and look?»; «If both are yes, leave it out.»
- ⊘ Ciò che deve **valere sempre** non è una riga: «Claude Code (the harness) has to execute this. This can be a hook, a deny rule, or a managed setting.» E una riga ignorata non si ripara con l'enfasi: «If Claude keeps ignoring a line that is in CLAUDE.md, adding IMPORTANT probably won't fix it.»

# La riga che resta, la riga che va

## Che cosa decide una riga, e dove si vede la seconda versione

- Nessuna delle tre richieste legge `legacy/` o il generatore, né tocca un file generato: per questo v2 contro v1 è «non distinguibile» in ogni criterio. La v2 **si vede altrove**
- La regola con `paths` si carica **quando serve**: «A path-scoped rule loads when Claude uses the Read, Write, or Edit tool on a matching file.» Il passo 6 del README: una sessione nuova legge un file di `legacy/`, o del generatore, e l'ultima riga del registro dell'hook dice:

| | il gestionale | la rete |
|---|---|---|
| `load_reason` | `path_glob_match` | `path_glob_match` |
| `file_path` | `.claude/rules/legacy.md` | `.claude/rules/generatore.md` |
| `trigger_file_path` | `progetti/gestionale-php/legacy/listino.inc.php` | `progetti/rete/generatore/topologia.py` |

L'ultima riga di .claude/caricamenti.jsonl, il registro dell'hook InstructionsLoaded del corso, in tabella
*Registrazione del corso, il percorso del partecipante, 7 novembre 2026*

- E il **peso del file**, nella terza richiesta, i token della prima chiamata al modello in base, v1 e v2: 4.879, 5.663 e 5.601 nel gestionale, dove la v2 ha tolto dal `CLAUDE.md` la riga di `legacy/`; 4.877, 5.351 e 5.352 nella rete, con lo stesso `CLAUDE.md`. Anthropic: «… each line in it is part of what every turn resends.»
- A voce: quale riga avete tolto perché Claude la trovava da solo, e quale avete aggiunto?

# La riga che resta, la riga che va

## Che cosa decide una riga, e dove si vede la seconda versione

- Nelle prove del docente, con `/init` e i due prompt del README, scritte da una sessione di Claude Code **senza nessuno davanti**, **nessuna riga è stata tolta** perché Claude la trovava da solo: la riga dei comandi è rimasta, perché con la v1 Claude ha speso meno turni. Il seguito lo chiede così: «Togli una riga se in base Claude ha fatto da solo ciò che chiede, e con la v1 non ha risparmiato turni.»
- Nel gestionale la v2 ha **spostato** la riga di `legacy/` dal `CLAUDE.md`, da 23 righe a 19, in `.claude/rules/legacy.md`, con `paths`; nella rete il `CLAUDE.md` è rimasto quello, e la v2 ha **aggiunto** la regola del generatore. Tutte e due hanno aggiunto le regole `deny` sui file generati
- Le regole `deny` scritte da Claude nel gestionale non hanno la **barra iniziale**, come `Edit(progetti/gestionale-php/database/dati/**)`: per la pagina dei permessi è un «Path relative to current directory». Gli esempi di riferimento la mettono

**Una riga resta se cambia ciò che Claude fa; se Claude la trova da solo, si toglie.**

# Gli esempi di riferimento

## Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E02, soluzione/CLAUDE.md, a capo per la slide, 1 di 2

```
# Gestionale CiaspoLink

Il gestionale PHP sta in `progetti/gestionale-php/`;
`tools/php.sh` esegue PHP, Composer e PHPUnit anche dove PHP non
c'è.

## Comandi

- Dalla cartella `progetti/gestionale-php/`:
  `../../tools/php.sh composer install` se manca `vendor/`, poi
  `../../tools/php.sh phpunit`.
```

- Il formato della documentazione: «Structure: group related instructions under markdown headers and bullets. Organized sections are easier for Claude to follow than dense paragraphs.»
- I **comandi**, con la cartella da cui si lanciano: «How to confirm work in this repo: the exact test, typecheck, and lint commands». Senza il file Claude li ha trovati, con il doppio dei turni
- Lo schema del database **non c'è**: sta in `database/schema.sql`, e Claude lo può leggere
- Esempi, non le sole versioni buone: le vostre sono diverse, e il controllo dice che cosa cambiano

# Gli esempi di riferimento

## Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E02, soluzione/CLAUDE.md, a capo per la slide, 2 di 2

```
## Convenzioni che il codice non dice

- Ogni commit ha nel messaggio una riga `Rif: GES-<n>`, il numero
  della richiesta nel tracker del team; `Rif: GES-0` per la
  manutenzione, come un refuso.
- Il soggetto segue `docs/convenzioni.md`, 11.1:
  `tipo(ambito): descrizione`, all'indicativo presente.

## Insidie

- `database/dati/` e `composer.lock` si rigenerano, non si
  modificano: i dati con `python3 genera.py`, le dipendenze con
  Composer.
```

- Le **decisioni** che il repository non ha, «Decisions that aren't in the repo»: la riga `Rif:`, decisa in una riunione. E la forma dei commit, che nel repository c'è, ma senza il file Claude non l'ha seguita
- Le **insidie**, con il comando che rimedia: «The root file should be pointers and critical gotchas only; everything else drifts into noise.»

# Gli esempi di riferimento

Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E02, .claude/rules/legacy.md, a capo per la slide

```
---
paths:
  - "progetti/gestionale-php/legacy/**"
---

# Il codice legacy

- In `legacy/` valgono solo le regole della sezione 9 di
  `docs/convenzioni.md`: leggile prima di cambiare un file.
- Si corregge un difetto o si sposta logica in `src/`; le
  funzionalità nuove vanno in `src/`.
- Prima di cambiare un comportamento, un test di caratterizzazione
  fissa il comportamento attuale.
- Una riformattazione va in un commit suo, mai insieme a una
  correzione.
```

- Si carica soltanto quando Claude legge, scrive o modifica un file di `legacy/`. Restano fuori il separatore del CSV, che il codice dice; la fatturazione, una procedura, da mettere in una skill; la preferenza personale, in un `CLAUDE.local.md`

# Gli esempi di riferimento

## Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E03, soluzione/CLAUDE.md, a capo per la slide, 1 di 2

```
# Rete CiaspoLink

Configurazioni, log, inventario e allarmi sintetici in
`progetti/rete/`, scritti da `genera.py` con il pacchetto
`generatore/`.

## Comandi

- «A posto» vuol dire i due controlli, dalla radice:
  `python3 progetti/rete/genera.py --check` e
  `python3 progetti/rete/genera.py --canone`; si riporta l'esito
  di ciascuno.
```

>_ Che cosa vuol dire **«a posto»**, con i due controlli e la cartella da cui si lanciano. Senza il file Claude li ha trovati, con un turno in più e più del triplo dei token

# Gli esempi di riferimento

## Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E03, soluzione/CLAUDE.md, a capo per la slide, 2 di 2

```
## Convenzioni che il codice non dice

- Il soggetto dei commit nella forma
  `rete(<ambito>): <descrizione>`, all'indicativo presente;
  l'ambito è la parte toccata: `audit`, `generatore`,
  `inventario`, `doc`.
- Ogni messaggio ha una riga `Rif: NOC-<n>`, il numero della
  segnalazione del NOC; `Rif: NOC-0` per la manutenzione, come un
  refuso.

## Insidie

- `configurazioni/`, `log/`, `inventario.csv` e `allarmi.csv` non
  si modificano a mano: si cambia `generatore/` e si rigenera.
```

La forma dei commit e la riga `Rif:` stanno soltanto negli appunti: «A decision your team made in a meeting»

# Gli esempi di riferimento

Il `CLAUDE.md` e la regola con `paths` di ogni traccia, nel formato di Anthropic

M6-E03, .claude/rules/generatore.md, a capo per la slide

```
---
paths:
  - "progetti/rete/generatore/**"
---

# Il generatore della rete

- Dopo una modifica a `generatore/`, rigenera dalla radice:
  `python3 progetti/rete/genera.py`.
- Poi i due controlli: `python3 progetti/rete/genera.py --check` e
  `python3 progetti/rete/genera.py --canone`.
- I file rigenerati vanno nello stesso commit della modifica al
  generatore.
```

- Come quella del gestionale, si carica quando Claude legge, scrive o modifica un file del generatore
- Le regole deny, come `Edit(/progetti/rete/log/**)`, fermano gli strumenti di file, non il generatore, che riscrive quei file: «They don’t apply to … arbitrary subprocesses that read or write files indirectly, like a Python or Node script that opens files itself.»